Focus Maps for Dense Administrative Boundaries

Screen-Space Selected-Area Inspection for Shiny and Web Mapping

George Arthur

Keywords: focus+context maps; administrative boundaries; Shiny; htmlwidgets; viewport transformation; interactive cartography; dense municipal maps

Working draft. Companion to the `explodemap` paper.

Abstract

Dense administrative maps are difficult to inspect interactively because small polygons compete for limited screen area, labels collide, hover events become visually noisy, and selection often requires repeated manual panning and zooming. This paper introduces a focus-map interaction model for dense administrative boundaries, implemented in the `focus_map()` widget in the `explodemap` R package. Unlike exploded-view cartography, which modifies feature positions in projected map coordinates, focus maps operate in viewport space: a selected feature is fit to a target screen fraction, visually lifted for emphasis, and paired with a non-blocking information card.

We formalize the interaction as a bounded camera transform driven by the selected feature's projected bounding box. Given a viewport of size (W, H) and selected feature screen-space extent (w_i, h_i), the model computes a scale factor that fits the selection within a target viewport fraction while respecting padding and maximum zoom constraints. We additionally define screen-space label visibility thresholds and an information-card placement model that preserves map interaction by avoiding blocking modal overlays. The resulting design supports municipal- and county-level Shiny workflows where users need fast selected-area inspection without losing map context.

Contents

1 Introduction 3

2 Relationship to Exploded-View Cartography 4

3 Notation 4

4 Selected-Feature Viewport Fitting 5

5 Lifted-Feature Emphasis 6

6 Label Visibility 6

7 Non-Blocking Information Cards 7

8 Implementation in R and Shiny 7

9 Dense-Layer Performance 8

10 Research Questions 8

11 Conclusion 9

1. Introduction

Interactive administrative maps often fail at the moment users need detail. A county-scale map may be readable at first glance, but municipal boundaries in dense metropolitan cores require repeated zooming, panning, tooltip hunting, and label management. In Shiny applications, this creates a practical problem: the map is not only a picture but a control surface. Users click or hover to retrieve attributes, compare units, and navigate the geography. When small features occupy only a few screen pixels, the interface becomes fragile.

The `explodemap` method addresses one version of this problem by transforming the map geometry: polygons are translated in projected coordinates to introduce whitespace while preserving feature shapes. The `focus_map()` widget addresses a different version of the problem. It keeps the map as an interactive object and changes the view around the user's current selection. Its goal is not to create a new static layout, but to provide a smooth selected-area inspection pattern for Shiny and web maps.

The central idea is simple: when a user selects an administrative unit, the widget computes the selected feature's screen-space extent, pans the camera to it, chooses a scale that makes the feature legible but not oversized, optionally lifts the feature for visual emphasis, and displays selected attributes in a non-blocking card. This creates a focus+context interaction: the selected polygon becomes readable while the surrounding map remains available for orientation and further interaction.

Contributions. This draft develops:

1. A screen-space mathematical model for selected-feature viewport fitting in dense polygon maps.
2. A bounded focus transform with tunable target size, padding, maximum zoom, and lifted-feature scale.
3. A label visibility model based on projected screen area and minimum screen dimensions.
4. A non-blocking information-card model for selected-area attributes in Shiny applications.
5. An R/htmlwidgets implementation exposed through `focus_map()`, `focusmapOutput()`, and `renderFocusmap()`.

2. Relationship to Exploded-View Cartography

The focus-map interaction is related to, but mathematically distinct from, the exploded-view displacement model. In the explodemap paper, each feature G_i is transformed in projected map coordinates by

$$G_i' = G_i + t_i, \tag{2.1}$$

where $t_i \in \mathbb{R}^2$ is a rigid translation vector derived from hierarchical centroid structure. The result is a new geometry layer.

In a focus map, the selected feature geometry remains G_i in the source map layer. The interaction applies a viewport transform

$$V_i = (c_i, z_i) \tag{2.2}$$

where c_i is the camera center and z_i is the camera zoom or scale chosen for the selected feature. The mathematical object is therefore a view state, not a new geographic dataset. This distinction is important for claims about geometry preservation: focus maps preserve source geometry because they do not rewrite it; explodemaps preserve individual feature geometry because they translate it rigidly.

3. Notation

Let $\mathcal{G} = \{G_1, \ldots, G_n\}$ be a collection of polygonal administrative units in a map coordinate system. Let $P_z : \mathbb{R}^2 \to \mathbb{R}^2$ denote the map projection and camera transformation from map coordinates to screen coordinates at zoom level z. For a feature G_i, define its screen-space geometry at zoom z as

$$S_i(z) = P_z(G_i). \tag{3.1}$$

Let

$$B_i(z) = \mathrm{bbox}(S_i(z)) = [x_i^-, x_i^+] \times [y_i^-, y_i^+] \tag{3.2}$$

be the screen-space bounding box of the selected feature. Its width and height are

$$w_i(z) = x_i^+ - x_i^-, \tag{3.3}$$

$$h_i(z) = y_i^+ - y_i^-. \tag{3.4}$$

Let the viewport have pixel dimensions (W, H). Let $q \in (0, 1)$ denote the target fraction of the viewport that the selected feature may occupy, and let $p_{\mathrm{pad}} \geq 0$ denote screen-space padding in pixels.

4. Selected-Feature Viewport Fitting

When the user selects feature i, the focus map computes a target camera state that places G_i near the center of the viewport and scales it to a legible size. The selected feature must fit within a padded rectangle of effective width and height

$$W_{\text{eff}} = qW - 2p_{\text{pad}}, \tag{4.1}$$

$$H_{\text{eff}} = qH - 2p_{\text{pad}}. \tag{4.2}$$

The effective dimensions must be positive; in implementation they are clamped to a small positive value when the viewport is tiny.

Given the feature's current screen-space size (w_i, h_i), the scale required to fit the feature within the target rectangle is

$$\sigma_i = \min\left(\frac{W_{\text{eff}}}{w_i}, \frac{H_{\text{eff}}}{h_i}\right). \tag{4.3}$$

If the map engine uses zoom levels rather than a continuous scale factor, the equivalent zoom update is

$$z_i^\star = z_0 + \log_2(\sigma_i), \tag{4.4}$$

where z_0 is the current zoom level. The final zoom is bounded:

$$z_i = \min(z_{\max}, \max(z_{\min}, z_i^\star)). \tag{4.5}$$

The target camera center is the selected feature centroid or point-on-surface:

$$c_i = \text{centroid}(G_i), \tag{4.6}$$

with point-on-surface preferred for irregular or multipart polygons when the centroid may fall outside the visible feature.

Algorithm 1: Selected-feature focus transform

Input: Selected feature G_i, current zoom z_0, viewport (W, H), target fraction q, padding p_{pad}, maximum zoom $z_{\max}$

Output: Camera state (c_i, z_i)

Project G_i to screen coordinates at z_0;
Compute bounding box size (w_i, h_i);
$W_{\text{eff}} \leftarrow qW - 2p_{\text{pad}}$;
$H_{\text{eff}} \leftarrow qH - 2p_{\text{pad}}$;
$\sigma_i \leftarrow \min(W_{\text{eff}}/w_i,\ H_{\text{eff}}/h_i)$;
$z_i^\star \leftarrow z_0 + \log_2(\sigma_i)$;
$z_i \leftarrow \min(z_{\max}, \max(z_{\min}, z_i^\star))$;
$c_i \leftarrow \text{centroid}(G_i)$ or point-on-surface;
return (c_i, z_i);

5. Lifted-Feature Emphasis

Viewport fitting makes the selected feature legible, but dense maps also benefit from a short-lived visual emphasis. Let $\ell > 0$ be the lift scale. For display only, the selected feature can be rendered as

$$\widetilde{S}_i = a_i + \ell\,(S_i - a_i)\,, \tag{5.1}$$

where a_i is a screen-space anchor, typically the feature's screen-space centroid or point-on-surface. This transform scales the selected feature around its own anchor in screen space. It is not written back to the underlying geographic object.

The implementation exposes this through `lift_scale`. Values slightly above 1 enlarge the focused feature while preserving context; very large values can hide boundaries outside the viewport and should be paired with larger `focus_padding` or a smaller `focus_size`.

6. Label Visibility

Labels in dense maps should appear when they can be read and disappear when they would produce clutter. The focus-map widget uses screen-space thresholds rather than map-unit thresholds because readability depends on pixels, not metres.

Let A_i^{screen} be the screen-space area of the selected or hovered feature, and let w_i^{screen} and h_i^{screen} be its screen-space bounding-box dimensions. A label is eligible when

$$A_i^{\text{screen}} \geq A_{\min}, \qquad w_i^{\text{screen}} \geq w_{\min}, \qquad h_i^{\text{screen}} \geq h_{\min}. \tag{6.1}$$

The package exposes these thresholds as `area_min`, `width_min`, and `height_min`. This rule

is intentionally simple: it is easy to explain, fast to evaluate, and predictable in Shiny applications where feature counts can be large.

7. Non-Blocking Information Cards

Hover tooltips are useful for quick identification but become insufficient when selected areas need multiple attributes. Modal dialogs provide more room but block map interaction. The focus-map model therefore uses a non-blocking information card anchored to a viewport corner.

Let C be a rectangular card of width w_C and height h_C, with scale parameter $\rho > 0$. If (w_C^0, h_C^0) is the base card size, then

$$(w_C, h_C) = \rho(w_C^0, h_C^0). \tag{7.1}$$

For a top-right placement with margin m, the card origin is

$$o_C = (W - w_C - m,\ m). \tag{7.2}$$

Analogous formulas define top-left, bottom-right, and bottom-left placements.

The card displays fields selected by the user:

$$I_i = \{(k, x_{ik}) : k \in K\}, \tag{7.3}$$

where K is the set of columns supplied through `info_cols`. Optional display names are supplied through `info_labels`. The card title is selected from `info_title` or falls back to the feature label column.

8. Implementation in R and Shiny

The user-facing R API is:

```r
focus_map(
  x,
  label_col = "NAME",
  group_col = "region",
  info_cols = c("population", "area"),
  focus_size = 0.76,
  focus_padding = 40,
  lift_scale = 1.16,
  info_card_scale = 1
)
```

The widget accepts an `sf` object, an `exploded_map` object, or a `grouped_exploded_map` object. In Shiny, it is used through:

```
ui <- fluidPage(
  focusmapOutput("map", height = "700px")
)

server <- function(input, output, session) {
  output$map <- renderFocusmap({
    focus_map(munis, label_col = "NAME", info_cols = c("county",
        "pop"))
  })
}
```

The htmlwidget maintains feature selection, camera state, hover labels, and information-card state on the client side, while Shiny integration exposes selected-feature events back to the server. This division is important for performance: the first focus transition should not require a round trip through the Shiny server.

9. Dense-Layer Performance

For municipal layers with thousands of polygons, interaction quality depends on the first selection and reset transitions. The widget uses three performance principles:

1. Keep focus transitions client-side once the data layer is loaded.
2. Reduce camera transition duration and label work for dense layers.
3. Simplify rendering geometry when the user requests performance-oriented display.

The `performance_mode` argument enables these heuristics explicitly, while `NULL` allows the widget to choose density-aware defaults. The appropriate threshold is partly empirical and should be evaluated across county, municipal, and national examples.

10. Research Questions

This paper is currently a methodological draft. The next empirical stage should evaluate:

1. Whether target viewport fraction q improves selected-feature recognition time compared with ordinary map zoom.
2. Whether non-blocking cards improve attribute lookup compared with hover-only tooltips and modal dialogs.
3. How lift scale ℓ affects boundary readability and spatial-context retention.

4. Whether dense-layer performance mode reduces first-selection latency for large municipal datasets without reducing interpretability.

11. Conclusion

Focus maps provide an interaction layer for dense administrative boundaries in R and Shiny. The method is deliberately modest: it does not claim to solve cartographic generalization or change geographic geometry. Instead, it formalizes a useful selected-area inspection pattern in screen space. By fitting selected features to a target viewport fraction, bounding zoom, using label visibility thresholds, and presenting non-blocking attribute cards, `focus_map()` turns dense maps into more useful interactive controls for applied mapping workflows.

References

Furnas, G. W. (1986). Generalized fisheye views. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 16–23.

Roth, R. E. (2013). Interactive maps: What we know and what we need to know. *Journal of Spatial Information Science*, 6, 59–115.

Heer, J. and Shneiderman, B. (2012). Interactive dynamics for visual analysis. *Communications of the ACM*, 55(4), 45–54.

Arthur, G. (2026). A hierarchical vector-based framework for multi-scale exploded-view cartography. Working manuscript.